What a Mutation Regime Can Know About a Function

Anonymous Author(s)
Affiliation

***Abstract*—Mutation testing measures a test suite by the fraction of a fixed family of *mutant* programs it distinguishes, and a score of one is read as evidence of adequacy. What such a score establishes about the program under test has never been characterized independently of a particular program, suite and mutant pool. We give that characterization for output mutation, which perturbs the returned value; extreme, Descartes-style mutation is its constant case. We ask when a full score against one regime guarantees a full score against another, for every program and every suite. For adequacy an output operator is exactly its footprint, the set of values it changes, and our main theorem (Theorem 3.2) answers the question: a *finite* regime Π is *sufficient* for a target class Γ iff every target footprint is the union of the Π-footprints it contains, while an infinite regime obeys a finite-avoidance form of the same law. The characterization is constructive: when it fails, its proof yields a program and a suite on which Π scores one and Γ does not. Its consequences locate what a passing score can *know* about a function. On n output values the minimal sufficient regime consists of the n singleton *value guards*, and Descartes constants are sufficient iff $n = 2$. A full score knows exactly output coverage: the suite exercised every value the program can return, and nothing more (Theorem 4.5). A finite absolutely sufficient regime exists only on finite return types, and deciding sufficiency is polynomial for value tables, decidable for Presburger-definable footprints and undecidable for computable operators (Theorem 5.2). Under a weak oracle the theory relativizes and characterizes the *pseudo-tested* method (§9). We separate three kinds of knowledge: whether a mutant changes the output, whether the suite's assertions see the change, and whether the change matters. Our theory settles the first two within its model; the third, the *intent residual*, needs a specification from outside the program (§10). Applying uniform output operators to the test suites of four Python libraries (131 functions, 985 mutants), we find 9% of the functions with a type-preserving constant pseudo-tested under the value oracle. Our model predicts that a constant mutant is detected exactly when one of its point changes is, which holds whenever a suite checks results call by call. That prediction held for every function where it was testable, except two whose type-changing constant crashes a test before its assertion (§9.5). Some sampled functions have side effects, so this tests the output-transformation prediction on real, partly stateful, executions rather than validating the pure-function model as a whole. The central characterization and its corollaries are mechanized in Lean 4 / Mathlib, and a theorem ledger identifies the results proved only on paper. Our artifact includes a decision procedure for finite sufficiency that returns the theorem's counterexample.**

***Index Terms*—Mutation testing, test adequacy, operator completeness, extreme mutation, interactive theorem proving.**

1. Introduction

1.1 The question

Take a function with three possible return values $\{a, b, c\}$ and a test suite whose tests happen to observe only a and b. Under extreme mutation, which replaces the body with `return a`, `return b` or `return c`, every mutant dies, because each constant differs from some observed output, so the suite scores one. Yet the fault "returns b where it should return c" survives, since no test ever sees c (Example 4.4). A perfect score against one operator family certified nothing about a fault that another family expresses.

This paper answers, for one class of operators, the question that example raises: *when does a full score against one mutation regime guarantee a full score against another, for every program and every suite?* We call this relation **sufficiency** (Definition 3.1). It is the precise form of a plainer question, namely *what does a passing score let you know about the function?*

For **output mutation**, operators that perturb a function's returned value regardless of how it was computed, the answer is exact. An operator matters to adequacy only through its **footprint**, the set of values it changes (Proposition 2.4). A finite regime Π is sufficient for a target class Γ iff every target footprint is the union of the Π-footprints it contains (Theorem 3.2), and an infinite regime obeys a finite-avoidance form of the same law (Theorem 3.2b). On three values the constants fail because no constant's footprint fits inside $\{c\}$. Our characterization is constructive: when it fails, its proof builds a program and a suite on which Π scores one and the target does not, and our artifact implements it as a decision procedure that returns this counterexample.

Why this is not a known result. Adequacy of a test set relative to a class of alternative programs is due to Budd and Angluin [3] and Howden [7]; both fix the program and its class, and neither relates adequacy for one class to adequacy for another. Three later results come closest to doing so, and §11 compares all of them precisely. Alshahwan and Harman [1] select tests whose outputs differ, their *output-uniqueness* criterion; it is a test selection criterion evaluated empirically. Our Theorem 4.5 shows that a full score against *all* output operators coincides with observing every reachable output, the absolute special case, whereas Theorem 3.2 is a statement about the relation between *any two* regimes, which output uniqueness leaves untouched. *Minimal* and *dominator* mutant sets (Ammann et al. [2]) are defined relative to a fixed test set and mutant pool, and true semantic subsumption, the order they approximate, is undecidable (Kurtz et al. [10]). *Sufficient* replacement sets for relational and conditional operators (Kaminski, Ammann and Offutt [9]; Just et al. [8]) are program-independent, but each is derived for one operator

schema from the truth table at the point of mutation. Theorem 3.2 is a necessary-and-sufficient characterization over the whole output-operator class, quantified over all programs and suites.

Program-independence comes from the object. A program-text mutation *schema* produces a mutant that depends on the *text* of the program rather than on the function it computes alone, so in general its detection varies across programs computing the same function, and uniform statements about it have to be earned schema by schema; the point-of-mutation results above earn them for relational and conditional operators from the operator's own truth table. Output operators act on the returned value alone, so their detection factors through the footprint for the whole class at once (Proposition 7.2). That general footprint reduction, rather than an impossibility result for other schemas, is what makes an exact theory available here.

1.2 The frame: three kinds of knowledge

A passing score bears on three different questions about a function, and our theory answers them to different extents.

- **Semantic distinguishability: does a mutant change the output?** This is $\mathrm{Mov}(p)$, and §§2–8 answer it completely under an exact oracle. A full score against all output operators is equivalent to output coverage (Theorem 4.5). Executions supply positive witnesses, while unresolved equivalent mutants prevent unconditional automation in general, even for Boolean outputs (§10).
- **Observational distinguishability: do the suite's assertions see the change?** This is the oracle-relative detection set $\mathrm{Det}_\omega(p)$ (§9). The difference $\mathrm{Mov}(p) \setminus \mathrm{Det}_\omega(p)$ is exactly what a given oracle hides, and it is what makes a covered method pseudo-tested.
- **Specification relevance: does the change matter?** Which of the distinctions the regime certifies bear on what the program is supposed to do is fixed neither by the program text nor by the oracle. We call it the **intent residual**. It requires a specification from outside the program: an authored intent test, a reference implementation or a formal contract (§10).

Within its model this paper characterizes the first two layers, and it locates the boundary to the third.

1.3 Contributions

1) **The sufficiency characterization** (§3; Theorems 3.2 and 3.2b, machine-checked): a finite regime Π is sufficient for Γ, uniformly over all programs and suites, iff every target footprint is the union of the Π-footprints it contains; arbitrary regimes satisfy the finite-avoidance form. It rests on the **footprint reduction** (§2): for adequacy an output operator *is* its footprint. Its proof is constructive, and our artifact turns it into a decision procedure with re-checkable counterexamples.

Every remaining result is a consequence or an extension of that characterization.

2) **The minimal sufficient regime and the ceiling** (§4): the n value guards are the minimal absolutely sufficient regime; constants are sufficient iff $n = 2$; a full score knows exactly output coverage.
3) **Existence and effective boundaries** (§5): finite absolute regimes require finite return types; deciding sufficiency is polynomial, decidable or undecidable according to the footprint representation. This is distinct from deciding program reachability or equivalent mutants.
4) **What structure alone leaves open** (§§6–7): coupling is not entailed for output mutation; detection factors through the footprint, which is why the theory is program-independent.
5) **The cost of the knowledge** (§8): the minimum certifying suite has $|f(D)|$ tests, the teaching dimension.
6) **The oracle-relative boundary** (§9): the development relativizes to a weak oracle and characterizes the pseudo-tested method under the mixed reading production tools use; §10 separates this observational layer from the intent residual.
7) **An experiment with the model's operators** (§9.5): uniform output operators applied to the test suites of four Python libraries, measuring pseudo-testedness and checking the output-transformation prediction on real execution traces, some of them from stateful code.

Our central characterization and the corollaries listed in the theorem ledger (§11) are mechanized in Lean 4 / Mathlib, stated over programs `f : D → R` and finite suites, and the ledger identifies the results proved only in this paper (§11).

2. The model

Definition 2.1 (program, suite, output operator). A program computes a total function — its *denotation* — $f : D \to R$, from inputs D to a return type R with $|R| \geq 2$. A *suite* is a finite $T \subseteq D$. An *output operator* is a total map $p : R \to R$; it produces the mutant $p \circ f$ (run the program, then perturb the result). A *regime* Π is a set of output operators.

Definition 2.2 (footprint). The *footprint* of p is $\mathrm{Mov}(p) = \{r \in R : p(r) \neq r\}$, the values it changes. Write $I = f(D)$ for the program's **reachable** outputs and $O = f(T) \subseteq I$ for the suite's **observed** outputs.

Definition 2.3 (equivalence, killing, score). $p \circ f$ is *equivalent* iff $p(f(x)) = f(x)$ for all $x \in D$; T *kills* it iff $p(f(x)) \neq f(x)$ for some $x \in T$; the score $\mathrm{score}_\Pi(f, T) = 1$ iff T kills every non-equivalent $p \circ f$, $p \in \Pi$.

Proposition 2.4 (an operator is its footprint). For any f, T, p:

$$p \circ f \text{ equivalent} \iff \mathrm{Mov}(p) \cap I = \varnothing,$$
$$T \text{ kills } p \circ f \iff \mathrm{Mov}(p) \cap O \neq \varnothing.$$

So whether p is a non-equivalent survivor depends only on $\mathrm{Mov}(p)$. Equal footprints are interchangeable, and $\mathrm{score}_\Pi(f, T)$ depends only on $\{\mathrm{Mov}(p) : p \in \Pi\}$.

Proof. $p \circ f$ is equivalent iff p fixes every element of $I = f(D)$, i.e. $\mathrm{Mov}(p) \cap I = \varnothing$. T kills $p \circ f$ iff some $x \in T$

has $f(x) \in \mathrm{Mov}(p)$, i.e. $\mathrm{Mov}(p) \cap f(T) = \mathrm{Mov}(p) \cap O \neq \varnothing$. Both read only $\mathrm{Mov}(p)$ against the fixed sets I, O. □

Remark 2.5 (every subset is a footprint). For $|R| \geq 2$ every $S \subseteq R$ is some operator's footprint: fix $a \neq b$, let $q(a) = b$ and $q(x) = a$ otherwise (fixed-point-free), and set $p(x) = q(x)$ for $x \in S$, $p(x) = x$ otherwise. So footprints range over the entire powerset, and a regime's adequacy content is exactly the family $\mathrm{Foot}(\Pi) = \{\mathrm{Mov}(p) : p \in \Pi\}$. *Where* an operator sends the values it moves is invisible to adequacy, and everything below rests on that fact.

The knowability question, stated. Proposition 2.4 defines non-equivalence by intersection with I, and observed killing by intersection with O. Running a finite suite observes the latter, and for a surviving mutant it leaves the former undecided. *What a passing score lets you know about* f is therefore a question about set systems, which §3 takes up.

Remark 2.6 (scope, and where the observational layer enters). §2–§8 assume an **exact output oracle**: a mutant is killed exactly when a test yields a different return value (Definition 2.3), so the suite observes the full output. This is the load-bearing assumption, and it is where the second kind of knowledge of §1.2 enters. Under a weaker oracle a *covered* mutant can survive because no assertion inspects the changed value, which is the pseudo-tested method, and §9 relativizes the whole theory to make this precise. Two simplifying assumptions remain: denotations are pure total maps (void methods, exceptions and effects are out of scope), and an operator perturbs every returned value identically (a statement-level mutation of one `return` site is not of this form).

3. The sufficient regime

Definition 3.1 (sufficiency). A regime Π is **sufficient for a target class** Γ iff for *every* program $f : D \to R$ and *every* finite suite T,

$$\mathrm{score}_\Pi(f, T) = 1 \implies \mathrm{score}_\Gamma(f, T) = 1.$$

Π is **absolutely sufficient** iff it is sufficient for $\Gamma = \mathcal{T}(R)$, all output operators. The quantifier over all (f, T) removes both the program-text and the suite relativity of prior completeness notions: sufficiency asks what a kill certifies *as such*, not on some corpus. Here D **varies**, since the quantifier ranges over all input domains and all $f : D \to R$. By Proposition 2.4 what matters is only the pair $(I, O) = (f(D), f(T))$ with $O \subseteq I$, and §11's `realizable` shows every such pair is met by an actual program, so quantifying over programs and over (I, O) pairs coincide. The separating program built in Theorem 3.2's proof uses a finite domain of size $|D| = |A| + 1 \leq |\Pi| + 1$, one escape point per operator that moves b, plus b itself.

Theorem 3.2 (footprint characterization). Let Π be finite. Then Π is sufficient for Γ iff

$$\forall g \in \Gamma,\ \forall b \in \mathrm{Mov}(g),\ \exists p \in \Pi : \quad b \in \mathrm{Mov}(p) \subseteq \mathrm{Mov}(g),$$

equivalently: **every target footprint is the union of the Π-footprints it contains**.

Proof. ($\Leftarrow$) Assume the condition; fix (f, T) with $\mathrm{score}_\Pi(f, T) = 1$ and non-equivalent $g \in \Gamma$. By Proposition 2.4 pick $b \in \mathrm{Mov}(g) \cap I$; take $p \in \Pi$ with $b \in \mathrm{Mov}(p) \subseteq \mathrm{Mov}(g)$. Then $b \in \mathrm{Mov}(p) \cap I$, so $p \circ f$ is non-equivalent, hence killed: $\mathrm{Mov}(p) \cap O \neq \varnothing$. Since $\mathrm{Mov}(p) \subseteq \mathrm{Mov}(g)$, also $\mathrm{Mov}(g) \cap O \neq \varnothing$, so T kills $g \circ f$.

($\Rightarrow$) By contraposition. Suppose the condition fails at $g \in \Gamma$, $b \in \mathrm{Mov}(g)$: every $p \in \Pi$ with $b \in \mathrm{Mov}(p)$ has an escape point $a_p \in \mathrm{Mov}(p) \setminus \mathrm{Mov}(g)$. Let $A = \{a_p\}$; finite (as Π is), with $b \notin A$. Take $I^\star = \{b\} \cup A$, $D = I^\star$, $f = \mathrm{id}$ (so $I = I^\star$), and $T = A$ (so $O = A$). Every non-equivalent Π-mutant dies. If $b \in \mathrm{Mov}(p)$ then $a_p \in \mathrm{Mov}(p) \cap O$. If $b \notin \mathrm{Mov}(p)$ then $\mathrm{Mov}(p) \cap I = \mathrm{Mov}(p) \cap A$, and this set is nonempty because $p \circ f$ is non-equivalent (Proposition 2.4); since $A = O$, the mutant is killed. But $g \circ f$ survives: $b \in \mathrm{Mov}(g) \cap I$ so it is non-equivalent, while $\mathrm{Mov}(g) \cap O = \mathrm{Mov}(g) \cap A = \varnothing$. So $\mathrm{score}_\Pi = 1 \not\Rightarrow \mathrm{score}_\Gamma = 1$. □

Theorem 3.2b (general Π). Theorem 3.2 uses the finiteness of Π only to keep the escape set A finite. Without it the containment $\mathrm{Mov}(p) \subseteq \mathrm{Mov}(g)$ weakens to a **finite-avoidance** condition. For *arbitrary* Π, Π is sufficient for Γ iff

$$\forall g \in \Gamma,\ \forall b \in \mathrm{Mov}(g),\ \forall F \subseteq R \setminus \mathrm{Mov}(g) \text{ finite},$$
$$\exists p \in \Pi :\ b \in \mathrm{Mov}(p)\ \wedge\ \mathrm{Mov}(p) \cap F = \varnothing.$$

For finite Π this reduces to Theorem 3.2 (take F to be the escape set A). Exactly on infinite families over an infinite R do the two forms differ. On $R = \mathbb{N}$ the regime $\Pi = \{\{0, k\} : k \geq 1\}$ is sufficient for the guard target $\mathrm{Mov}(g) = \{0\}$, because for any finite $F \subseteq \mathbb{N} \setminus \{0\}$ some $k \notin F$ gives $\{0, k\} \cap F = \varnothing$; yet Π contains no footprint $\subseteq \{0\}$, so Theorem 3.2's condition *fails* while 3.2b's *holds*. Theorem 3.2 is therefore the finite-Π instance of the general law, and its finiteness hypothesis cannot be dropped. Both forms are machine-checked (`footprint_characterization`, `footprint_characterization_general`; §11).

Remark 3.3 (composition is the wrong closure). Theorem 3.2 dissolves the temptation to define sufficiency by *generation under composition*. Killing a regime's operators need not kill their composites. On $R = \{0, 1, 2\}$ with sole reachable/observed value 0, and $a(0) = 1$, $b(0) = 2$, $b(1) = 0$ (extended arbitrarily to the rest of R), both $a \circ f$ and $b \circ f$ die at 0, yet $(b \circ a)(0) = b(1) = 0$, so $(b \circ a) \circ f$ is equivalent and unkilled. Composition acts on exactly the part of an operator that adequacy cannot see, since $\mathrm{Mov}(b \circ a)$ can be strictly smaller than both factors' footprints. So "Π generates the transformation monoid" says nothing about what a score certifies. The governing order is footprint containment, not compositional generation.

4. The minimal sufficient regime, and the ceiling

Corollary 4.1 (the value-guard basis; minimal size n). Let $|R| = n$. A regime Π is absolutely sufficient iff it contains, for

every $r \in R$, an operator with footprint $\{r\}$ — a *value guard* γ_r, "if the result is r, return something else." The minimal absolutely-sufficient regime is $\{\gamma_r : r \in R\}$, of size exactly n.

Proof. Absolute sufficiency is sufficiency for $\mathcal{T}(R)$, whose footprints are all of 2^R (Remark 2.5). Apply Theorem 3.2b to the singleton target $\mathrm{Mov}(g) = \{b\}$ with $F = R \setminus \{b\}$, which is finite because R is: sufficiency forces some $p \in \Pi$ with $b \in \mathrm{Mov}(p)$ and $\mathrm{Mov}(p) \cap (R \setminus \{b\}) = \varnothing$, i.e. $\mathrm{Mov}(p) = \{b\}$, a value guard for b. (Because R is finite, the whole complement is an admissible F, so no finiteness hypothesis on Π is needed here.) Conversely a guard per value satisfies 3.2b's condition for every target. Each of the n singleton targets forces a distinct guard, so no smaller regime suffices. □

Corollary 4.2 (mechanical sufficiency requires a finite return type). A finite regime can be absolutely sufficient only when R is finite. For an infinite return type, such as `int`, `String` or objects, *no finite output-mutation regime is absolutely sufficient*.

Proof. Let Π be finite and absolutely sufficient. For each $b \in R$, Theorem 3.2 applied to the singleton target $\{b\}$ yields some $p \in \Pi$ with $b \in \mathrm{Mov}(p) \subseteq \{b\}$, i.e. $\mathrm{Mov}(p) = \{b\}$. As $r \mapsto \{r\}$ is injective, $|\Pi| \geq |R|$, so R is finite. □

Corollary 4.2 bounds **finite absolutely sufficient operator families**, not effective knowledge of reachability. Such families exist on finite return types: Booleans, enums, small tagged unions. On the return types of most real programs the absolute regime is unavailable, so the meaningful object is *relative* sufficiency for a chosen footprint class (§5), which is the certificate a tool can actually offer.

Corollary 4.3 (extreme mutation is optimal on Booleans, deficient beyond). A constant operator `return c` has footprint $R \setminus \{c\}$. The constant regime is absolutely sufficient iff $n = 2$: then `return true`/`return false` have footprints $\{\text{false}\}$/$\{\text{true}\}$, exactly the two guards, while for $n \geq 3$ every constant footprint has size ≥ 2, so no constant is a guard.

Example 4.4 (a distinction the constants cannot know). Let $R = \{a, b, c\}$, a program reaching all three, a suite observing only $\{a, b\}$. All three constants die, so the constant score is 1. But the fault "returns b where it should return c", with footprint $\{c\}$, is non-equivalent ($c \in I$) and unkilled ($\{c\} \cap O = \varnothing$). A perfect extreme-mutation score determines nothing about the c-outputs the suite never observed.

Theorem 4.5 (the ceiling — exactly what a full score knows). For $|R| \geq 2$ and any f, T:

$$\mathrm{score}_{\mathcal{T}(R)}(f, T) = 1 \iff I \subseteq O \iff O = I,$$

i.e. an absolute score of one holds **exactly when the suite exercises every reachable output**. This is the whole of what output mutation determines, and no more.

Proof. ($\Leftarrow$) If $O = I$, every non-equivalent g has $\mathrm{Mov}(g) \cap I \neq \varnothing$, hence $\mathrm{Mov}(g) \cap O \neq \varnothing$, so all die. ($\Rightarrow$) If $I \not\subseteq O$, the guard γ_b for $b \in I \setminus O$ is non-equivalent and unkilled, so the score is below one. □

Remark 4.6 (the ceiling, and the first face of the intent residual). Theorem 4.5 is the operational content of output mutation. A full score certifies **output coverage**, that every value the program can return was observed, and the value guards are the cheapest regime making that exact. It knows nothing about faults separating two inputs mapped to the same output: if $f(x_1) = f(x_2)$ then $p(f(x_1)) = p(f(x_2))$ for every output operator, so no output mutant distinguishes them. Input-separating behavior is structurally outside the regime's knowledge, the first and purely structural piece of what the mechanical layer cannot reach.

5. Where knowability is decidable

Fix a finite Π and a finitely-presented target Γ. By Theorem 3.2, deciding sufficiency is deciding footprint containment, so its complexity is exactly that of the footprint representation.

Proposition 5.1 (the exact reach of a partial guard regime). The guards $\{\gamma_{r_1}, \ldots, \gamma_{r_k}\}$ are sufficient for exactly $\Gamma_{\max} = \{g : \mathrm{Mov}(g) \subseteq \{r_1, \ldots, r_k\}\}$.

Proof. By Theorem 3.2 the guards are sufficient for g iff every $b \in \mathrm{Mov}(g)$ has a guard γ_{r_i} with $b \in \{r_i\} \subseteq \mathrm{Mov}(g)$, i.e. $b \in \{r_1, \ldots, r_k\}$; this holds for all $b \in \mathrm{Mov}(g)$ iff $\mathrm{Mov}(g) \subseteq \{r_1, \ldots, r_k\}$. □

Proposition 5.1 is the usable form. A partial guard regime yields an explicit, checkable certificate: *every guarded value the program can produce is exercised, hence any fault confined to the guarded values is caught*, with the guarded set a named parameter.

Theorem 5.2 (decidability spectrum). Let Π and Γ be finite. By Theorem 3.2, Π is sufficient for Γ iff

$$\forall g \in \Gamma \;\; \forall b \in \mathrm{Mov}(g) \;\; \exists p \in \Pi : \\ b \in \mathrm{Mov}(p) \;\wedge\; \mathrm{Mov}(p) \subseteq \mathrm{Mov}(g),$$

so its decidability is that of this condition for the representation in which operators are given:

1) **Value tables.** R is finite with $|R| = n$, and each operator is given as its table of n values, so the input has size $N = (|\Pi| + |\Gamma|)\, n$. Sufficiency is decidable in time $O(|\Pi|\,|\Gamma|\, n) \subseteq O(N^2)$.
2) **Presburger-definable footprints.** $R = \mathbb{Z}$, and each operator is given by a Presburger formula $\varphi_p(x)$ defining $\mathrm{Mov}(p)$. Sufficiency is decidable.
3) **Total computable operators.** $R = \mathbb{N}$, and each operator is given as a program computing a total function $\mathbb{N} \to \mathbb{N}$, so each footprint is a decidable set. Sufficiency is undecidable; indeed the set of sufficient instances is not recursively enumerable.

Proof. (1) Compute each footprint by one pass over its table. For each $g \in \Gamma$, scan Π once to form $U_g = \bigcup\{\mathrm{Mov}(p) : p \in \Pi,\ \mathrm{Mov}(p) \subseteq \mathrm{Mov}(g)\}$, at $O(n)$ per operator, and test $\mathrm{Mov}(g) \subseteq U_g$; the displayed condition is equivalent

to $\mathrm{Mov}(g) \subseteq U_g$ for every g. (2) With Π and Γ finite, the displayed condition is a single sentence of Presburger arithmetic,

$$\bigwedge_{g\in\Gamma} \forall b \left(\varphi_g(b) \to \bigvee_{p\in\Pi}(\varphi_p(b) \wedge \forall y \left(\varphi_p(y) \to \varphi_g(y)\right))\right),$$

whose truth is decidable (Presburger [14]). (3) Appendix A gives a computable map from Turing machines M to instances $(\{h\}, \{g_M\})$ of total computable operators such that $\{h\}$ is sufficient for $\{g_M\}$ iff M never halts on empty input. The non-halting machines form a set that is not recursively enumerable, so neither is the set of sufficient instances. □

Remark 5.3 (the spectrum runs through set containment). The P / decidable / undecidable gradient is that of subset containment for increasingly expressive set representations. No word problems or Cayley embeddings appear, because adequacy never composes operators (Remark 3.3). It is the containment order, and not a generation order, that governs whether knowability is decidable. So the second knowability boundary, whether the sufficiency question itself is answerable, is exactly the decidability of footprint containment.

6. What structure alone cannot determine

Proposition 6.1 (coupling is not entailed). There exist a regime Π, program f, and suite T under which every non-equivalent first-order Π-mutant is killed while a non-equivalent higher-order mutant survives.

Proof. Let $R = \{0, 1, 2\}$, $\Pi = \{a\}$ with $a : 0 \mapsto 1, 1 \mapsto 0, 2 \mapsto 1$ (so $\mathrm{Mov}(a) = R$), and f with $I = \{0, 2\}$, $O = \{0\}$. The first-order mutant $a \circ f$ is killed at 0. But $a \circ a : 0 \mapsto 0, 1 \mapsto 1, 2 \mapsto 0$ has $\mathrm{Mov}(a \circ a) = \{2\}$: non-equivalent ($2 \in I$), unkilled ($2 \notin O$). So the score is 1 against $\{a\}$ and below 1 against its composition closure. □

The coupling-effect hypothesis (DeMillo, Lipton and Sayward [4]), that killing first-order mutants tends to kill higher-order ones, is therefore not entailed for output mutation. Its mechanism is the one in Remark 3.3: a composite's footprint can be strictly smaller than its factors'. By Theorem 3.2 a higher-order mutant is simply an operator with a footprint, and the composition closure may contain footprints the regime leaves uncovered. The hypothesis is a statistical claim about typical faults. Proposition 6.1 shows that it is no theorem here, and nothing more than that; it says nothing against the hypothesis in practice.

7. Why the theory is program-independent

Proposition 7.1 (program-independent subsumption). For output operators p, g: *every test that kills $p \circ f$ also kills $g \circ f$, for every program f*, iff $\mathrm{Mov}(p) \subseteq \mathrm{Mov}(g)$.

Proof. ($\Leftarrow$) A test x kills $p \circ f$ iff $f(x) \in \mathrm{Mov}(p) \subseteq \mathrm{Mov}(g)$, so it kills $g \circ f$. ($\Rightarrow$) If $b \in \mathrm{Mov}(p) \setminus \mathrm{Mov}(g)$, a program reaching b with a suite observing b kills $p \circ f$ but not $g \circ f$. □

For finite Π, Theorem 3.2 reads: **for every target g and every moved value $b \in \mathrm{Mov}(g)$, some $p \in \Pi$ both moves b and subsumes g uniformly over programs**. Its pointwise coverage clause is essential, since one subsuming mutant per target is insufficient. For example, footprints $\mathrm{Mov}(p) = \{0\}$ and $\mathrm{Mov}(g) = \{0, 1\}$ satisfy subsumption, but with $I = \{1, 2\}$ and $O = \{2\}$ the regime $\{p\}$ has full score while g survives non-equivalently. Subsumption without coverage of every moved value fails to establish sufficiency.

Proposition 7.2 (the factoring that buys program-independence). For an output operator the detection set factors through the footprint: $\mathrm{Det}(p \circ f) = \{x : p(f(x)) \neq f(x)\} = f^{-1}(\mathrm{Mov}(p))$. So p enters detection only through the program-independent object $\mathrm{Mov}(p)$. The contrast with program-text mutation is subtler than syntax against semantics, since any mutant's detection set is semantic. A program-text schema's mutant depends on the *text* of f, so in general its detection is a function of the (schema, source) pair rather than of the schema alone, and it can vary across programs computing the same f. Particular schemas still admit program-independent statements, as the relational and conditional operator results do at the point of mutation (§11), but each must be derived for its schema. What output operators have is a general reduction: every operator in the class is characterized by its footprint, which is why the knowability theory of §3 is available for the whole class at once.

8. The cost of the knowledge

Theorem 8.1 (minimum certifying suite). Fix an absolutely-sufficient regime and a program f with reachable set $I = f(D)$. A suite T achieves $\mathrm{score}(f, T) = 1$ iff $O = I$. **If I is finite**, the minimum certifying suite has size $\sigma(f) = |I| = |f(D)|$ (one test per reachable value). **If I is infinite**, no finite suite is certifying: finite T has finite $O = f(T)$, so $O \neq I$.

Proof. By Theorem 4.5 the score is 1 iff $O = I$. If I is finite, a suite with $O = I$ has, for each $r \in I$, some x with $f(x) = r$, so $|T| \geq |I|$; one x per value gives $|T| = |I|$. If I is infinite, $O = f(T)$ is finite for finite T. □

Remark 8.2 (the sample complexity of knowledge). $\sigma(f)$ is the **teaching dimension** of f with respect to the concept class $\mathcal{C}_\Pi(f) = \{f\} \cup \{p \circ f : p \in \Pi\}$, i.e. the size of a smallest teaching set for f in $\mathcal{C}_\Pi(f)$ (Goldman and Kearns [6]): the minimum number of labeled examples, tests with their observed outputs, that distinguish f from every non-equivalent mutant the regime expresses. For output mutation it is exactly the number of reachable outputs. Theorem 3.2 says *which* faults a regime can certify against, and Theorem 8.1 says *how many tests* the certification costs. So the mechanically-knowable profile of a function's output behavior is both characterized and *priced*.

9. The oracle-relative boundary and the pseudo-tested method

Our model's one load-bearing simplification is the exact oracle (Remark 2.6), and it is also the one that most bounds practical relevance, because the phenomenon that motivates extreme mutation, the *pseudo-tested method*, is a weak-oracle

phenomenon by construction. This section relativizes the theory to an arbitrary oracle and characterizes exactly what a weak oracle hides. §10 then separates that observational layer from the intent residual.

Definition 9.1 (oracle). An *oracle* is a map $\omega : R \to \Omega$ recording what the suite's assertions observe about a returned value. The exact oracle is $\omega = \mathrm{id}$; a suite checking only a field, a hash, a type tag, or nothing is a non-injective ω.

Definition 9.2 (oracle-relative detection). T *ω-kills* $p \circ f$ iff $\omega(p(f(x))) \neq \omega(f(x))$ for some $x \in T$; $p \circ f$ is *ω-equivalent* iff equality holds for all x. The oracle-relative detection set is $\mathrm{Det}_\omega(p) = \{r : \omega(p(r)) \neq \omega(r)\}$.

Proposition 9.3 (the relativized reduction). $p \circ f$ is ω-equivalent iff $\mathrm{Det}_\omega(p) \cap I = \varnothing$, and T ω-kills it iff $\mathrm{Det}_\omega(p) \cap O \neq \varnothing$. Also $\mathrm{Det}_\omega(p) \subseteq \mathrm{Mov}(p)$, with equality for injective ω. The set-system sufficiency results (Theorems 3.2 and 3.2b) and subsumption characterization (Proposition 7.1) therefore apply to the actual families of Det_ω sets. The singleton-guard ceiling and its suite-size consequence require a **nonconstant oracle**: only then can every singleton detection set be realized by an output operator. Effective decidability statements additionally require the stated effective representation of Det_ω itself; a simple Mov representation alone does not ensure it for arbitrary ω. With constant ω, all mutants are ω-equivalent, every score is vacuously one, and every regime is absolutely ω-sufficient, independently of output coverage.

Proof. Identical to Proposition 2.4 reading $\omega \circ p$ against ω; containment is immediate. □

Remark 9.3b (the footprint abstraction partially gives way). Under a non-injective ω, $\mathrm{Det}_\omega(p)$ reads $\omega \circ p$, so *where* p sends the values it moves, invisible to adequacy under the exact oracle (Remark 2.5), becomes visible again: two operators with equal Mov can have different Det_ω. Corollary 4.3 is where this bites. A constant has $\mathrm{Det}_\omega(\text{`return } c\text{'}) = R \setminus \omega^{-1}(\omega(c))$. For **finite** R and **nonconstant** ω, the constant regime is absolutely ω-sufficient iff $|R| = 2$. On two values a nonconstant ω necessarily separates them ($\omega(\text{true}) \neq \omega(\text{false})$), so each constant's detection set is the other value's singleton; for $|R| \geq 3$, a singleton detection set $R \setminus \omega^{-1}(\omega(c)) = \{b\}$ for every b would put all of R in one ω-class, i.e. make ω constant. So "extreme mutation is optimal on Booleans" survives relativization for every oracle that inspects the boolean at all, and fails only for the constant oracle, under which every score is vacuously one (Proposition 9.3).

Where Mov and Det_ω differ lies the interesting content. Under a weak oracle the ceiling stays where it was: for any non-constant ω and any value b there is an operator with $\mathrm{Det}_\omega(p) = \{b\}$, so the absolute ω-score is 1 iff $I \subseteq O$. Output coverage remains exactly what a full score knows, provided the suite asserts *something*. What weakening the oracle changes is *which regimes* reach the ceiling, and that is what makes a covered mutant survive.

Proposition 9.4 (pseudo-testedness, characterized). Let f be covered by T (so $O \neq \varnothing$). The constant mutant `return c` is **pseudo-tested** — it changes a value the program genuinely returns, yet no test in T detects the change under oracle ω — iff

$$\mathrm{Mov}(\text{`return } c\text{'}) \cap I \neq \varnothing \quad \text{and} \quad \mathrm{Det}_\omega(\text{`return } c\text{'}) \cap O = \varnothing,$$

equivalently $I \neq \{c\}$ (some reachable output is not c, so the constant is non-equivalent under the **exact** oracle) and $O \subseteq \omega^{-1}(\omega(c))$ (every observed output is oracle-indistinguishable from c, so the **weak** oracle sees no change).

Here the crux is that the two conditions read non-equivalence and survival under *different* oracles. Non-equivalence must be exact-oracle. A paradigm pseudo-tested method is one whose output no assertion inspects at all, so *every* reachable output lies in c's ω-class ($I \subseteq \omega^{-1}(\omega(c))$) and the mutant is *$\omega$-equivalent*; yet its return value genuinely differs from the original on some input ($\mathrm{Mov} \cap I \neq \varnothing$), which is exactly the fault the weak oracle fails to catch. Requiring ω-non-equivalence, the natural but wrong reading $\mathrm{Det}_\omega \cap I \neq \varnothing$, would exclude this case, the very phenomenon the notion names. Under the exact oracle $\omega = \mathrm{id}$ we have $\mathrm{Det}_\omega = \mathrm{Mov}$ and the two conditions collapse to $O = \{c\}$ against $I \neq \{c\}$: the suite observes only c while the program returns more.

Proof. ω-survival is $\mathrm{Det}_\omega(\text{`return } c\text{'}) \cap O = \varnothing$, i.e. $O \subseteq \omega^{-1}(\omega(c))$. Exact-oracle non-equivalence is $\mathrm{Mov}(\text{`return } c\text{'}) \cap I \neq \varnothing$; since $\mathrm{Mov}(\text{`return } c\text{'}) = R \setminus \{c\}$ this is $I \neq \{c\}$. Under $\omega = \mathrm{id}$, $\mathrm{Det}_\omega = \mathrm{Mov}$ and $\omega^{-1}(\omega(c)) = \{c\}$. □

This is the model's account of the empirical phenomenon. A method is pseudo-tested exactly when the suite's assertions collapse everything it observes into one oracle class, so replacing the body with a constant in that class changes nothing the suite can see. Mechanically, the regime establishes that the returned value changed ($\mathrm{Mov} \cap I \neq \varnothing$), and the gap $\mathrm{Mov}(p) \setminus \mathrm{Det}_\omega(p)$ is exactly what this oracle hides. Closing it takes an assertion that pins the value. Whether the value *should* be pinned, that is, whether the hidden distinction matters, is a question about the specification rather than the oracle (§10). This account also explains why extreme mutation is a productive *detector* even though the constant regime is a weak *certifier* (Corollary 4.3): the two roles differ, and the surfaced gap is a candidate assertion the suite is missing.

Proposition 9.4b (the mixed ceiling — the score a tool actually reports). Definition 9.2 discounts ω-equivalent mutants, so the absolute ω-score of the paragraph above is not the number a tool reports. Production tools (Descartes, PIT) count a mutant as non-equivalent when it changes the returned value, the *exact* oracle, and as killed when the suite notices, the *weak* oracle ω, which is exactly the mixed reading Proposition 9.4 adopts. Applied to the ceiling, that reading gives a strictly stronger theorem. The absolute **mixed** score (exact-oracle non-equivalence, ω-kills) is 1 iff

$$O = I \quad \text{and} \quad \omega^{-1}(\omega(b)) = \{b\} \ \text{ for every } b \in I,$$

i.e. the suite observes every reachable output *and* the oracle separates every reachable value from every other. Under the exact oracle the second clause is automatic ($\omega = \mathrm{id}$ is injective) and this collapses to Theorem 4.5.

Proof. ($\Leftarrow$) Let p be non-equivalent under the exact oracle: some $b \in \mathrm{Mov}(p) \cap I$. Then $b \in O = I$ and $p(b) \neq b$, and $\omega^{-1}(\omega(b)) = \{b\}$ forces $\omega(p(b)) \neq \omega(b)$, so T ω-kills p at b; hence every exact-non-equivalent mutant is ω-killed. ($\Rightarrow$) If $b \in I \setminus O$, the guard γ_b is exact-non-equivalent and ω-unkilled (its only moved value b is unobserved), so it survives. If some $b' \neq b$ shares b's ω-class with $b \in I$, the operator sending $b \mapsto b'$ is exact-non-equivalent yet has $\mathrm{Det}_\omega = \varnothing$, so it survives. Either failure drops the score below one. □

Remark 9.4c (the scope of an output oracle). Definition 9.1 lets an assertion depend only on the returned value. Real assertions can also depend on the input and on other calls, and two cases fall outside the model in different ways.

- *Input-dependent checks.* Suppose $f(x_1) = f(x_2) = r$ and the suite asserts the value returned for x_1 but never inspects the one for x_2. Detection is then a per-call oracle ω_x rather than a single ω on R, so $\mathrm{Det}_\omega(p) \cap O$ no longer determines killing and Proposition 9.3's reduction no longer applies as stated. Detection of a mutant is still the union of per-call events, one for each executed call whose result the mutant changes.
- *Joint checks.* An assertion over several results, such as `assert f(x1) != f(x2)`, is not even a union of per-call events. With $f(x_1) \neq f(x_2)$, the constant mutant makes the two results equal and is detected, while a mutant that changes only one of them, to that same constant, is not.

We formalize neither generalization. §9.5 measures how often real suites stay within the per-call form, using a prediction that holds for every per-call oracle.

9.5 Empirical: uniform output operators on four Python libraries

Our theory makes two predictions a test suite can be held to. First, whether a covered function is pseudo-tested depends on the oracle (Proposition 9.4). Second, under the model a constant mutation is detected exactly when one of its point changes is. Write $s_{r\to c}$ for the **point swap** that sends the value r to c and fixes everything else. For observed outputs O,

$$\mathrm{Det}_\omega(\text{`return } c\text{'}) \cap O \;=\; \bigcup_{r \in O \setminus \{c\}} \big(\mathrm{Det}_\omega(s_{r\to c}) \cap O\big),$$

so `return c` is killed iff some $s_{r\to c}$ with $r \in O$ is killed (Proposition 9.3). This holds for any *per-call* oracle as well (Remark 9.4c), provided changing one call's result leaves unchanged which other calls occur: the constant changes exactly the calls the swaps change between them, and each call's detection event is the same under both. So agreement with this prediction measures how often real suites stay within per-call checking, and a disagreement identifies a check outside it.

What this tests, and what it does not. The theory of §§2–8 assumes pure total functions. Our experiment applies the theory's operator, post-composition on the returned value, to real functions, and some of the sampled functions are impure: inflect's `def*` methods register a user-defined inflection, and boltons' `LRI.setdefault` writes to a cache. For such functions the prediction still applies to the recorded execution traces, as long as perturbing a returned value leaves the calls and side effects the tests observe unchanged. So the study tests the output-transformation prediction on real execution traces, some of them from stateful programs; it validates no more of the pure-function adequacy model than that.

Design. We pinned four libraries to release commits: boltons 26.2.0, inflect 7.6.0, toolz 1.2.0 and humanize 4.16.0. Each candidate is a public function or method in a public module, and it is mutated as the model prescribes. An injected decorator, innermost on the function, returns $p(f(x))$ in place of $f(x)$ at the outermost call, so recursive calls inside f stay unperturbed. Each project's own suite is used, including its doctests where the project runs them. A test is *usable* if it passed twice with the decorator installed and inactive, and also on the unmodified checkout. A recording run logs every returned value of every candidate, per test. A function is *eligible* if some usable test calls it, every observed output has a stable representation (no memory addresses), and it returns something other than `None` (void functions are outside the model, Remark 2.6). We drew a seeded sample of at most 40 eligible functions per library. For each sampled function, c is a type-appropriate constant (`0`, `""`, `[]`, `{}`, `False` and so on) chosen so that some observed output differs from it. When no single built-in type covers the outputs, c is `None`, and we report that subset separately. Each function receives the constant mutant `return c` and the swaps $s_{r\to c}$ for every observed $r \neq c$, sampled down to 12 where there are more. Each mutant runs only the function's covering usable tests. A test failure counts under the **value oracle** when the test's own check rejects the result: an `AssertionError` raised in test code, `pytest.fail` or `pytest.raises` failing, or a doctest output mismatch. It counts under the **any-failure oracle** when the test fails in any way, including an exception from library code or a timeout.

Of 646 candidates, 336 were eligible: 147 were not covered by a usable test, 94 returned values without a stable representation (mostly iterators and generators in toolz), and 69 returned only `None`. We sampled 131 (40, 40, 34 and 17) and ran 985 mutants: 131 constants and 854 swaps, against 1,853 usable tests. Every constant mutant was applied at least once. Nine tests behave differently with the decorator installed and are excluded from the usable set. In boltons, four tests pickle classes made by factories that read the caller's frame, or inspect frames directly. In inflect, five tests check `typeguard` runtime validation, which does not instrument a function wrapped by our decorator.

TABLE I
COVERED FUNCTIONS PSEUDO-TESTED UNDER THE UNIFORM CONSTANT MUTATION, BY ORACLE (§9.5). PARENTHESIZED: THE SUBSET WHOSE CONSTANT PRESERVES THE RETURN TYPE.

Library	Functions	Pseudo-tested (value)	Pseudo-tested (any failure)
boltons	40 (30)	7 (3)	1 (1)
inflect	40 (35)	6 (5)	5 (5)
toolz	34 (16)	1 (0)	0 (0)
humanize	17 (16)	1 (1)	0 (0)
Pooled	**131 (97)**	**15 (9)**	**6 (6)**

Parenthesized counts are the 97 functions whose constant preserves the return type.

Pseudo-testedness. Under a uniform type-preserving constant, 9 of 97 sampled functions (9%) are pseudo-tested under the value oracle and 6 (6%) under the any-failure oracle; over all 131 the figures are 15 (11%) and 6 (5%). Functions whose constant only a crash detected make a paired difference of 3.1 percentage points for the type-preserving subset (95% bootstrap interval [0.0, 7.2], resampling functions) and 6.9 points over all 131 ([3.1, 11.5]). Per-library value-oracle rates (3–18%) fall within the 1–46% that Vera-Pérez et al. [18] report across 21 Java projects, although their operators and per-method definition differ from ours. All six functions pseudo-tested under both oracles have return values that no test inspects: five inflect `def*` methods, which register a user-defined inflection and return `1`, and boltons' `LRI.setdefault`. All six are stateful, and none of their covering tests detects a change to the returned value.

The prediction. With the any-failure oracle the prediction held for every function where it can be tested: 86 of 86 functions with a complete set of swaps, and 44 of 44 with a sampled set in which some swap was killed (on a sampled set only "some swap killed ⇒ the constant killed" is testable). One function's outputs vary between runs, so not every swap was applied; it is counted as untestable rather than as a swap that went undetected. With the value oracle the prediction held for 85 of 86 and 41 of 42. Both exceptions are boltons functions whose constant is `None`: the constant makes a test crash before reaching the assertion that catches the individual swaps, so the value oracle never sees it. That is the type artifact the type-preserving subset avoids, and in that subset the prediction held without exception: 59 of 59 complete, and 36 of 36 (value) and 37 of 37 (any failure) sampled. No joint check of the kind in Remark 9.4c surfaced in this sample.

Threats to validity.

- *The direction of the gap is fixed by construction.* Any-failure kills include value kills, so only the gap's magnitude carries information, and a crash can pre-empt an assertion (the two value-oracle exceptions).
- *The value oracle is a family.* A value-pinning `==` is close to $\omega = \mathrm{id}$, while a check on a field, a length or a type is a strictly weaker ω; the value column mixes them.
- *Reachable outputs are approximated by observed ones.* Non-equivalence of each constant is guaranteed, since c differs from some observed output, but the swaps cover only observed values.
- *Value identity is by representation.* Swaps match values by `repr`, which excludes 94 candidates and makes swaps on nondeterministic outputs unreliable. We count each operator's applications and treat unapplied swaps as missing.
- *Some sampled functions are stateful.* The model assumes pure functions; for methods with side effects the study tests the prediction on the observed traces only (see "What this tests" above).
- *The decorator has side effects.* It excludes nine tests, and in inflect it removes `typeguard`'s runtime checks from decorated methods in every run.
- *The sample is small.* Four Python libraries and 131 functions bear on how often real suites fit the model's form, not on how prevalent pseudo-testing is in general.

Pinned commits, environment lists, seeds, per-test outcome records and the analysis script are all in the artifact.

10. THE INTENT RESIDUAL

Our results separate three kinds of knowledge about a function's output behavior (§1.2). The theory determines the first two within its model, and the third needs a source outside the program.

Semantic distinguishability, and the evidence for it. The ceiling theorem characterizes a semantic score that excludes genuinely equivalent mutants, without providing a procedure for recognizing those mutants. For a singleton guard γ_b, an observed kill proves $b \in O$, whereas a surviving guard proves only $b \notin O$. Two possibilities remain: $b \notin I$ (equivalent) or $b \in I \setminus O$ (reachable but unobserved). A witnessed execution reaching b resolves that ambiguity positively, and a justified equivalence proof can resolve it negatively; unsuccessful search alone cannot.

A finite return type makes the absolute guard family finite (Corollary 4.2), and leaves reachability undecidable. For any machine M, the total Boolean function $f_M(n)$ that returns whether M halts within n steps reaches `True` iff M halts. Thus a tool computing all reachable outputs, or deciding equivalence of the `True` guard, would decide halting. Theorem 5.2 concerns sufficiency of represented operator footprints rather than equivalence of their compositions with arbitrary programs. Full output coverage is mechanically decidable on a finite, exhaustively executable input domain, or under additional reachability certificates. Otherwise the honest operational result records observed kills, proven equivalences, and unresolved candidate-equivalents separately. A certificate modulo those candidates is conditional on their equivalence; it is weaker than the unconditional score of Definition 2.3. Theorem 8.1 prices a certifying suite when it exists, without giving a general algorithm to find it.

Observational distinguishability. What the suite's assertions see is the oracle-relative detection set (§9). For each mutant, $\mathrm{Mov}(p) \setminus \mathrm{Det}_\omega(p)$ is exactly the set of changed values this oracle hides, and Proposition 9.4 characterizes when that

hides a whole constant mutation. This is a property of the pair (program, oracle), and the model determines it fully: given the reachable outputs and the oracle, nothing about it is left open. Its scope is an oracle that reads each returned value on its own (Remark 9.4c).

Specification relevance: the intent residual. Neither layer says whether a distinction *matters*. An oracle blind to a change is no evidence that the change is harmless. An author may intend two outputs to be interchangeable, in which case leaving the distinction unchecked is correct, and an assertion may detect a difference the specification ignores. Which of the certified distinctions bear on what the program is supposed to do is fixed neither by the program text nor by its oracle. We call this the *intent residual*. Two parts of it are structural: faults that separate inputs mapped to the same output lie outside output mutation entirely (Remark 4.6), and a hidden distinction $\mathrm{Mov}(p) \setminus \mathrm{Det}_\omega(p)$ is resolved only by deciding whether the value should be pinned. Resolving either takes a specification supplied from outside the program and its suite: an authored intent test stating that on this input the returned value *is* what it should be, a reference implementation, or a formal contract. Our theory leaves open which source must supply it; what it shows is that the program and its oracle cannot.

The knowability boundary therefore runs neither between easy and hard nor between cheap and expensive. It runs between output behavior that is observed or proved, semantic equivalence that remains unresolved, and intent that the program text leaves open. This paper's theorems characterize the first completely. That the other two are finite and named is what makes the division a workflow rather than a fog.

11. Related work, scope, and mechanization

Related work. The claim that mutation testing "lacks a completeness theorem" needs a precise qualifier, so the table compares the nearest results on four questions: what object each characterizes, what it quantifies over, whether it is necessary and sufficient, and whether it holds independently of a particular program and suite.

Underneath this paper lies a formal notion older than mutation tools. Budd and Angluin [3] call a test set T *adequate* for P relative to a neighborhood $\Phi(P)$ of alternative programs if every alternative is either equivalent to P or fails on some test. With $\Phi(P) = \{p \circ f : p \in \Pi\}$ that is exactly a score of one against Π. Howden [7] defines the same notion function by function. There T is *effective* for f relative to an error class S_f if any $f' \in S_f$ that agrees with f on T agrees with it everywhere, and *complete* if it is effective for every component function a method targets. He uses the definition to compare weak, strong and trace mutation, branch, domain and functional testing. Both papers fix the program and its class of alternatives. Budd & Angluin ask whether an adequate set exists and whether one can be recognized or generated: every finite neighborhood has one, existence is undecidable in general, and computable recognizers are tied to decidable program equivalence. Neither paper relates adequacy for one class to adequacy for another, which is the question Definition 3.1 asks over all programs and suites: Π is sufficient for Γ iff adequacy relative to $\{p \circ f : p \in \Pi\}$ implies adequacy relative to $\{g \circ f : g \in \Gamma\}$ for every f and T. Howden reports the nearest earlier result of that shape, from Brooks: for shape-preserving errors in a restricted class of recursive Lisp programs, a test set complete for single trace mutations is complete for repeated ones. That is a coupling theorem for one program class, whereas Theorem 3.2 characterizes sufficiency for the whole output-operator class. Budd & Angluin's existence question also has a sharp answer here: against all output operators a finite adequate suite exists iff $f(D)$ is finite (Theorem 8.1).

True semantic subsumption, the order a completeness theorem would rank against, is undecidable (Kurtz et al. [10]). Later constructive results are relative by construction: sufficient operators are found experimentally against a pool (Offutt et al. [13]), while minimal and dominator sets are test-set- and pool-relative (Ammann et al. [2]). Shin and Bae [16] give the closest formal framework: it represents the behavioral differences of mutants over a given set of tests as vectors and recovers mutant-set minimization and mutation-based fault localization within it. It is offered as a framework rather than a theory with theorems, and its object is a fixed test set and mutant pool. Program-independent results do exist, namely the sufficient replacement sets for relational and conditional operators (Kaminski, Ammann and Offutt [9]; Just et al. [8]): from a clause's truth table they show that a few replacements subsume the rest **at the point of mutation, for any predicate in any program**. That is the same kind of program-independent statement the present theory makes, for one operator schema at a time. Theorem 3.2 makes it for the whole output-operator class, as an exact characterization rather than a sufficient set. Likewise the coupling effect is more than an assumption. Wah's [19] analyses establish it under restricted fault models, so the honest statement is "unproven in general," not "never proven."

Two of this paper's constructs are prior art under other names. Theorem 4.5's ceiling, that a full score means every reachable output was observed, is the coverage notion behind the **output-uniqueness** criterion of Alshahwan and Harman [1], reached here as a theorem about footprints rather than posited as a criterion. Our value/oracle gap of §9 is **checked coverage** (Schuler and Zeller [15]), the existing dynamic measure of whether an assertion actually constrains a computed value. Extreme mutation and pseudo-testedness are due to Niedermayr et al. [12] and Vera-Pérez et al. [18], with the Descartes tool (Vera-Pérez et al. [17]). We recover extreme mutation as the $n = 2$ optimum (Corollary 4.3; Remark 9.3b for weak oracles) and characterize pseudo-testedness in the model's own terms (Proposition 9.4), and we take the teaching-dimension reading from Goldman and Kearns [6].

What is new. An exact characterization, for finite and for arbitrary regimes, of uniform sufficiency between output-operator families, expressed through their semantic footprints (Definition 3.1; Theorems 3.2 and 3.2b), with a constructive counterexample whenever it fails. No result in the table implies

TABLE II
PRIOR RESULTS COMPARED ON FOUR QUESTIONS.

Work	Object characterized	Quantified over	Necessary and sufficient	Program- and suite-independent
Howden [7]	effective and complete test sets: T is effective for f relative to an error class S_f if agreeing with f on T forces agreement everywhere	a given program, its component functions and their error classes	a definition, used to compare testing methods	no: a given program
Budd and Angluin [3]	adequacy of T for P relative to a neighborhood $\Phi(P)$: every alternative is equivalent to P or fails a test	a given program and neighborhood	existence and computability of adequate sets; undecidable in general	no: a given program and neighborhood
Offutt et al. [13]	a sufficient set of mutation operators	an experimental pool	no: found experimentally	no: pool-relative
Ammann et al. [2]	minimal and dominator mutant sets	a fixed test set and mutant pool	exact for that pool	no
Kurtz et al. [10]	true (semantic) mutant subsumption	all tests	undecidable	—
Kaminski, Ammann and Offutt [9]; Just et al. [8]	sufficient replacements for ROR and for COR	all truth-table rows of one clause	sufficient and minimal, per schema	yes, at the point of mutation, one schema each
Alshahwan and Harman [1]	output uniqueness, a test-selection criterion	—	no: evaluated empirically	—
Shin and Bae [16]	behavioral differences of programs as vectors over a test set	a given test set and mutant set	a framework, not a theorem	no
This paper	**sufficiency between any two output-operator families**	**all programs and all finite suites**	**yes (Theorems 3.2, 3.2b)**	**yes**

it: output uniqueness concerns the absolute case only and is no theorem; the point-of-mutation sufficiency results fix one operator schema and give a sufficient set rather than a characterization; and the subsumption results are relative to a test set and pool. Its consequences are the value-guard basis (Corollary 4.1), the ceiling (Theorem 4.5), the decidability spectrum (Theorem 5.2), the cost (Theorem 8.1) and the oracle relativization (§9). Proposition 6.1 clarifies Remark 3.3 rather than standing as an independent contribution. Two companion manuscripts, withheld here for review, develop what this paper's residual points to: one the effect/meaning frame and the decidable-coupling result, the other the teacher-supplied negative sign.

Scope. Our theory concerns output/extreme mutation of pure total functions. §§2–8 assume an exact oracle, which §9 removes. It stops short of general program-text mutation and makes no claim about the coupling hypothesis for program-text faults. Its effect layer has an undecidability of its own, general footprint containment (Theorem 5.2(3)), which we characterize rather than dissolve. Our central quantitative result, that a full score equals output coverage, is deliberately modest: it says what a score *means*, and leaves open whether a tool should compute it in preference to measuring output coverage directly.

Mechanization. The development is in Lean 4 (de Moura and Ullrich [5]) with Mathlib (the mathlib Community [11]), stated directly over programs `f : D → R` and finite suites `T : Finset D` and bridged to the footprint level (`proofs/adequacy_completeness.lean`, 38 theorems, no `sorry`). An audit file pins every theorem's axioms with `#guard_msgs`, so the build fails if any theorem depends on `sorryAx` or on an axiom beyond `propext`, `Classical.choice` and `Quot.sound`. Every numbered claim then receives one of four statuses in the ledger below: *mechanized* (the statement is a Lean theorem), *via lemmas* (a direct instantiation or assembly of checked theorems), *paper proof*, or *empirical*.

`THEOREM_LEDGER.md` in the artifact holds the full ledger, with every supporting lemma.

Reproducibility. Our artifact contains three independently runnable parts. Its Lean development builds from a fresh checkout with two commands (`lake exe cache get`, `lake build`) under the pinned toolchain (Lean 4 v4.28.0, Mathlib v4.28.0) in about two minutes on a laptop, running the axiom audit as part of the build. A standard-library Python implementation of Theorem 3.2 decides finite sufficiency and returns the necessity proof's counterexample, which an independent checker re-verifies from the definitions. Its tests compare the decision with the definition exhaustively on three-valued return types. For the §9.5 experiment, the artifact pins each library to a release commit, fixes its sampling seed, and ships every per-test record from which its numbers are computed.

12. CONCLUSION

For one operator class we replace a heuristic, a high score read as adequacy on the strength of the coupling hypothesis, with a theorem about what the score determines. The mechanically knowable profile of a function's output behavior is exactly characterized: which regime suffices to certify it (Theorem 3.2), the minimal such regime (Corollary 4.1), the exact content of a full score (Theorem 4.5, output coverage),

TABLE III
THEOREM LEDGER: HOW EACH CLAIM IS ESTABLISHED.

Paper	Claim	Status	Lean name
Prop 2.4	operator ≡ footprint, over programs	mechanized	`progScore_iff_scoreAt`
Remark 2.5	every subset is a footprint	mechanized	`mov_surjective`
Remark 3.3	composition shrinks footprints	mechanized	`composition_shrinks_footprint`
Thm 3.2	**footprint characterization (finite Π), over programs**	**mechanized**	`progComplete_characterization`
Thm 3.2b	general Π (finite avoidance)	mechanized	`footprint_characterization_general`
Cor 4.1	value-guard basis	mechanized	`progComplete_absolute_iff_guards`
Cor 4.2	no finite absolute regime on infinite R	mechanized	`progComplete_univ_infinite`
Cor 4.3	constants sufficient iff n = 2	mechanized	`constants_iff_card_two`
Example 4.4	constants on three values	mechanized	`constants_insufficient_on_three`
Thm 4.5	the ceiling	mechanized	`ceiling`
Remark 4.6	equal outputs are inseparable	paper proof	—
Prop 5.1	reach of partial guards	mechanized	`partial_guards_reach`
Thm 5.2	reduction to containment	via lemmas	Thm 3.2
Thm 5.2 (1)–(3)	P / decidable / undecidable	paper proof	Appendix A
Prop 6.1	coupling not entailed	mechanized	`coupling_fails`
Prop 7.1	subsumption = containment	mechanized	`subsumes_iff_subset`
§7 example	one subsuming footprint is insufficient	mechanized	`one_subfootprint_is_insufficient`
Prop 7.2	detection factors	mechanized	`det_factors`
Thm 8.1	minimum certifying suite	mechanized	`prog_certify_lb`, `prog_certify_ub`, `certify_infinite`
Remark 8.2	teaching dimension	paper proof	—
Prop 9.3	oracle-relative reduction	via lemmas	`detW_singleton_of_nonconstant`, `detW_surjective`, `detW_const`
Remark 9.3b	constants under a weak oracle	mechanized	`constants_omega_iff_card_two`
Prop 9.4	pseudo-testedness (mixed oracle)	mechanized	`pseudo_tested_mixed_iff`
Prop 9.4b	the mixed ceiling	mechanized	`mixed_ceiling`
§9.5	uniform-operator study	empirical	`experiment/`
§10	a surviving guard need not be a gap	mechanized	`surviving_guard_without_gap`
§10	Boolean reachability is undecidable	paper proof	—

where that knowledge is achievable (finite types) and decidable (footprint containment), and what it costs ($|f(D)|$ tests). Relative to an oracle, the theory also says exactly which changes the suite's assertions hide. What it leaves open is which of the certified distinctions bear on what the program is supposed to do; that intent residual takes a specification from outside the program and its oracle. For this class a mutation score is thus an object with a characterized semantics rather than a quantity trusted empirically. The line between what a tool can know about a function and what requires a specification is drawn exactly, and the central characterization is checked by machine.

APPENDIX A
UNDECIDABILITY OF SUFFICIENCY FOR COMPUTABLE OPERATORS

This appendix proves Theorem 5.2(3). Operators are total functions $\mathbb{N} \to \mathbb{N}$ given as programs; an instance is a pair of finite lists of such programs, (Π, Γ).

Construction. Fix the operator $h(k) = k+1$ if k is odd and $h(k) = k$ otherwise. It is total, and $\mathrm{Mov}(h) = \{k : k \text{ odd}\}$. For a Turing machine M run on empty input, let

$$g_M(k) = \begin{cases} k+1 & \text{if } M \text{ halts within } k \text{ steps,} \\ k & \text{otherwise,} \end{cases}$$

The operator g_M is total: computing $g_M(k)$ simulates M for at most k steps and always terminates. A program for g_M is obtained computably from a description of M, so $M \mapsto (\{h\}, \{g_M\})$ is a computable map into instances of total computable operators. Its footprint is $\mathrm{Mov}(g_M) = \{k : M \text{ halts within } k \text{ steps}\}$. Halting within k steps is monotone in k, so $\mathrm{Mov}(g_M) = \varnothing$ if M never halts, and $\mathrm{Mov}(g_M) = [t, \infty)$ if M halts after exactly t steps.

Claim. $\{h\}$ is sufficient for $\{g_M\}$ iff M never halts.

Proof. By Theorem 3.2 with the single operator h, sufficiency holds iff every $b \in \mathrm{Mov}(g_M)$ satisfies $b \in \mathrm{Mov}(h) \subseteq \mathrm{Mov}(g_M)$. If M never halts, $\mathrm{Mov}(g_M) = \varnothing$ and the condition holds vacuously. If M halts after t steps, $\mathrm{Mov}(g_M) = [t, \infty)$ contains an even number b, and $b \notin \mathrm{Mov}(h)$, so the condition fails at b. □

The set of machines that never halt on empty input is not recursively enumerable, and the map above reduces it to the set of sufficient instances; hence that set is not recursively enumerable, and in particular sufficiency is undecidable for total computable operators. The construction uses only Theorem 3.2, which is mechanized; the reduction itself is not.

REFERENCES

[1] Alshahwan, N., Harman, M.: Coverage and fault detection of the output-uniqueness test selection criteria. In: ISSTA, pp. 181–192 (2014). doi:10.1145/2610384.2610413

[2] Ammann, P., Delamaro, M.E., Offutt, J.: Establishing theoretical minimal sets of mutants. In: ICST, pp. 21–30 (2014). doi:10.1109/ICST.2014.13

[3] Budd, T.A., Angluin, D.: Two notions of correctness and their relation to testing. Acta Informatica **18**(1), 31–45 (1982). doi:10.1007/BF00625279

[4] DeMillo, R.A., Lipton, R.J., Sayward, F.G.: Hints on test data selection: help for the practicing programmer. IEEE Computer **11**(4), 34–41 (1978). doi:10.1109/C-M.1978.218136

[5] de Moura, L., Ullrich, S.: The Lean 4 theorem prover and programming language. In: CADE, LNCS 12699, pp. 625–635 (2021). doi:10.1007/978-3-030-79876-5_37

[6] Goldman, S.A., Kearns, M.J.: On the complexity of teaching. J. Comput. Syst. Sci. **50**(1), 20–31 (1995). doi:10.1006/jcss.1995.1003

[7] Howden, W.E.: Weak mutation testing and completeness of test sets. IEEE Trans. Softw. Eng. **SE-8**(4), 371–379 (1982). doi:10.1109/TSE.1982.235571

[8] Just, R., Kapfhammer, G.M., Schweiggert, F.: Do redundant mutants affect the effectiveness and efficiency of mutation analysis? In: ICST, pp. 720–725 (2012). doi:10.1109/ICST.2012.162

[9] Kaminski, G., Ammann, P., Offutt, J.: Improving logic-based testing. J. Syst. Softw. **86**(8), 2002–2012 (2013). doi:10.1016/j.jss.2012.08.024

[10] Kurtz, B., Ammann, P., Offutt, J.: Static analysis of mutant subsumption. In: ICST Workshops (Mutation), pp. 1–10 (2015). doi:10.1109/ICSTW.2015.7107454

[11] The mathlib Community: The Lean mathematical library. In: CPP, pp. 367–381 (2020). doi:10.1145/3372885.3373824

[12] Niedermayr, R., Juergens, E., Wagner, S.: Will my tests tell me if I break this code? In: CSED@ICSE, pp. 23–29 (2016). doi:10.1145/2896941.2896944

[13] Offutt, A.J., Lee, A., Rothermel, G., Untch, R.H., Zapf, C.: An experimental determination of sufficient mutant operators. ACM Trans. Softw. Eng. Methodol. **5**(2), 99–118 (1996). doi:10.1145/227607.227610

[14] Presburger, M.: Über die Vollständigkeit eines gewissen Systems der Arithmetik ganzer Zahlen, in welchem die Addition als einzige Operation hervortritt. In: Comptes Rendus du I Congrès de Mathématiciens des Pays Slaves, pp. 92–101. Warsaw (1929)

[15] Schuler, D., Zeller, A.: Checked coverage: an indicator for oracle quality. Softw. Test. Verif. Reliab. **23**(7), 531–551 (2013). doi:10.1002/stvr.1497

[16] Shin, D., Bae, D.H.: A theoretical framework for understanding mutation-based testing methods. In: ICST, pp. 299–308 (2016). doi:10.1109/ICST.2016.22

[17] Vera-Pérez, O.L., Monperrus, M., Baudry, B.: Descartes: a PITest engine to detect pseudo-tested methods (tool demonstration). In: ASE, pp. 908–911 (2018). doi:10.1145/3238147.3240474

[18] Vera-Pérez, O.L., Danglot, B., Monperrus, M., Baudry, B.: A comprehensive study of pseudo-tested methods. Empir. Softw. Eng. **24**(3), 1195–1225 (2019). doi:10.1007/s10664-018-9653-2

[19] Wah, K.S.H.T.: An analysis of the coupling effect I: single test data. Sci. Comput. Program. **48**(2–3), 119–161 (2003). doi:10.1016/S0167-6423(03)00022-4